

DSIP MODULE 4

Image Segmentation & Boundary Descriptors

■ Speed-Mode Revision Sheet ■

One-day-before-exam comprehensive review

Concepts • Memory Tricks • Worked Numericals • Practice Problems

#	Topic	Type
1	Image Segmentation - Intro	Concept
2	Point Detection	Concept + Numerical
3	Line Detection	Concept + Numerical
4	Edge Detection (Ideal vs Ramp)	Concept
5	Gradient Operators (Roberts/Prewitt/Sobel)	Concept + Numerical ■
6	Laplacian and LoG (Mexican Hat)	Concept
7	Edge Linking - Local Processing	Concept
8	Canny Edge Detector	Concept ■
9	Hough Transform - Both Forms	Numerical ■■■■
10	Thresholding (Global / Adaptive / Optimal)	Concept
11	Region-Based Segmentation Basics	Concept
12	Region Growing	Numerical ■■■■
13	Region Splitting (Quadtree)	Numerical ■■■■
14	Region Merging / Split-Merge	Concept
15	Chain Code	Numerical ■
16	First Difference	Numerical ■
17	Shape Number	Numerical ■
18	Signature	Concept
19	Image Moments (Area, Centroid)	Numerical ■

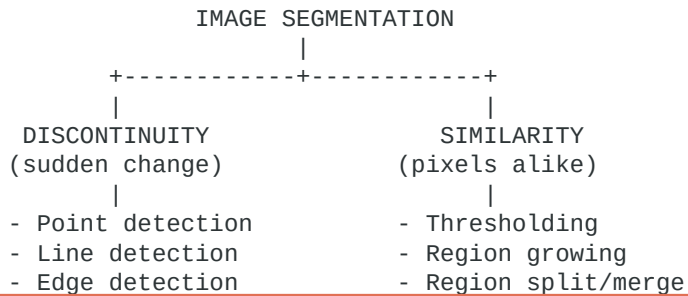
20	Master Cheat Sheet	Quick Recap
----	--------------------	-------------

1. Image Segmentation - Introduction

Definition: Image segmentation divides an image into regions where pixels *inside* a region are similar, and *adjacent* regions are different.

Goal: Find individual objects in an image.

Two Fundamental Approaches



■ **MEMORY TRICK:** "Discontinuity finds **BORDERS**, Similarity finds **INTERIORS**." Like a country: discontinuity = drawing border lines, similarity = coloring the territory.

The Universal Mask Formula

Every detection method (point, line, edge) works the same way: **slide a 3x3 mask over the image, compute weighted sum.**

Mask:	Image neighborhood:
+---+---+---+	+---+---+---+
w1 w2 w3	z1 z2 z3
+---+---+---+	+---+---+---+
w4 w5 w6	z4 z5 z6
+---+---+---+	+---+---+---+
w7 w8 w9	z7 z8 z9
+---+---+---+	+---+---+---+

Response: $R = w1z1 + w2z2 + \dots + w9z9 = \text{Sum}(w_i * z_i)$

■ **TIP:** The mask determines what you detect. Change the weights -> change what you find. That's it.

2. Point Detection

Mask & Rule

+---+---+---+	
-1 -1 -1	
+---+---+---+	
-1 8 -1	(sum of weights = 0)
+---+---+---+	
-1 -1 -1	
+---+---+---+	

Detection Rule: $|R| \geq T \rightarrow \text{Point detected}$

■ **MEMORY TRICK:** "Center 8, neighbors -1. Sum = 0. **Quiet on flat areas, screams at lonely pixels.**" Sum = 0 means uniform regions give zero response. Big +8 vs -1 surrounding means isolated pixels stand out.

■ **NUMERICAL:** Point Detection Example

Q: Apply point detection on this neighborhood with $T = 100$. Is the center an isolated point?

```
+-----+-----+-----+
| 10 | 10 | 10 |
+-----+-----+-----+
| 10 | 200 | 10 |
+-----+-----+-----+
| 10 | 10 | 10 |
+-----+-----+-----+
```

Solution:

$$R = (-1)(10) + (-1)(10) + (-1)(10) \\ + (-1)(10) + (8)(200) + (-1)(10) \\ + (-1)(10) + (-1)(10) + (-1)(10)$$

$$R = -80 + 1600 = 1520$$

$|R| = 1520 \geq 100 \rightarrow \text{YES, isolated point detected.}$

■ NUMERICAL: Practice Problem (already solved)

Q: Apply point detection with $T = 50$:

```
+-----+-----+-----+
| 20 | 22 | 18 |
+-----+-----+-----+
| 19 | 25 | 21 |
+-----+-----+-----+
| 20 | 23 | 19 |
+-----+-----+-----+
```

$$R = -20 - 22 - 18 - 19 + 200 - 21 - 20 - 23 - 19 = 38$$

$|R| = 38 < 50 \rightarrow \text{NOT a point.}$

Reason: center (25) too similar to neighbors.

3. Line Detection

The Four Direction Masks

HORIZONTAL	+45	VERTICAL	-45
+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+
-1 -1 -1	-1 -1 2	-1 2 -1	2 -1 -1
+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+
2 2 2	-1 2 -1	-1 2 -1	-1 2 -1
+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+
-1 -1 -1	2 -1 -1	-1 2 -1	-1 -1 2
+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+	+-----+-----+-----+

■ MEMORY TRICK: "+2 along the line direction, -1 everywhere else. Sum = 0." Look at where the 2s form a line - that is the orientation that mask detects.

- Horizontal $\rightarrow$ 2s in middle row
- Vertical $\rightarrow$ 2s in middle column
- +45 $\rightarrow$ 2s on anti-diagonal (bottom-left to top-right)
- -45 $\rightarrow$ 2s on main diagonal (top-left to bottom-right)

Usage: Apply all 4 masks. The mask with the largest $|R|$ tells you the line direction.

■ NUMERICAL: Line Detection Example

Q: Which line direction does this neighborhood contain?

```
+-----+-----+-----+
```

```

| 0 | 0 | 0 |
+---+---+---+
|100|100|100|
+---+---+---+
| 0 | 0 | 0 |
+---+---+---+

```

Solution:

Horizontal mask: $R = 0 + 200+200+200 + 0 = 600 \quad \leftarrow \text{WINS}$

Vertical mask: $R = 0$

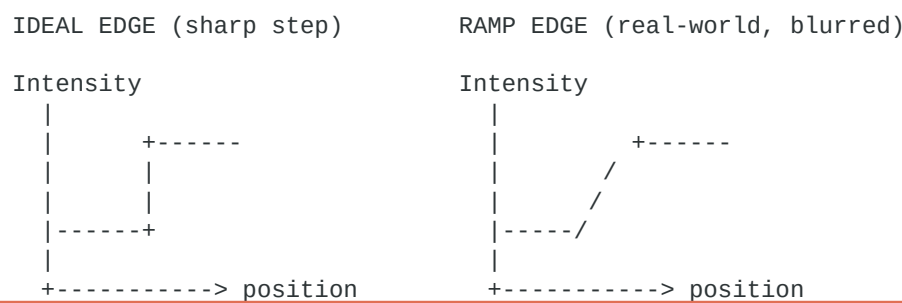
+45 mask: $R = 0$

-45 mask: $R = 0$

Largest $|R| = 600$ (horizontal mask) $\rightarrow$ HORIZONTAL line.

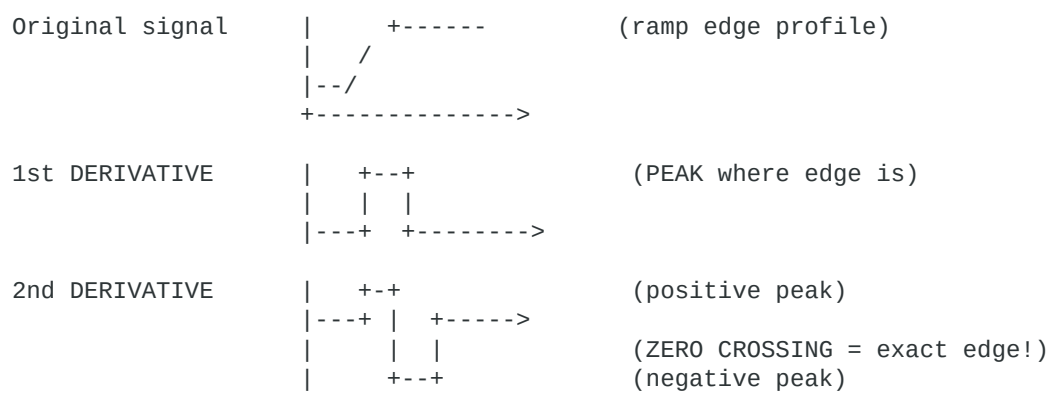
4. Edge Detection - Fundamentals

Two Edge Models



■ **MEMORY TRICK:** "Ideal = staircase. Ramp = wheelchair ramp." Real images have ramp edges due to blur, focus, sampling. Slope of ramp = how blurry the edge is. Steeper = sharper edge.

The Derivative Insight (CORE CONCEPT)



Key Properties Comparison:

Property	1st Derivative	2nd Derivative
Edge location	Peak (max value)	Zero crossing
Operators	Sobel, Prewitt, Roberts	Laplacian, LoG
Edge thickness	Thicker	Thinner (precise)
Noise sensitivity	Less sensitive	Very sensitive
Direction info	YES	NO (just magnitude)

■ **MEMORY TRICK:** "1st derivative - find the PEAK. 2nd derivative - find where it CROSSES ZERO." Picture a hill: 1st deriv = steepness, 2nd deriv = change in steepness. At cliff edge, change in steepness flips from + to - = zero crossing.

The Gradient Vector

$$\nabla f = [G_x, G_y] = [\partial f / \partial x, \partial f / \partial y]$$

$$\text{Magnitude: } |\nabla f| = \text{sqrt}(G_x^2 + G_y^2) \approx |G_x| + |G_y|$$

$$\text{Direction: } \alpha(x,y) = \text{tan}^{-1}(G_y / G_x)$$

■ **TIP:** In exam, use $|G_x| + |G_y|$ approximation - it's MUCH faster and accepted.

5. Gradient Operators (Roberts, Prewitt, Sobel)

Roberts (2x2, simplest)

Gx	Gy
+-----+-----+	+-----+-----+
-1 0	0 -1
+-----+-----+	+-----+-----+
0 1	1 0
+-----+-----+	+-----+-----+
(main diagonal)	(anti-diagonal)

Prewitt (3x3)

Gx	Gy
+-----+-----+	+-----+-----+
-1 -1 -1	-1 0 1
+-----+-----+	+-----+-----+
0 0 0	-1 0 1
+-----+-----+	+-----+-----+
1 1 1	-1 0 1
+-----+-----+	+-----+-----+

Sobel (3x3, most popular)

Gx	Gy
+-----+-----+	+-----+-----+
-1 -2 -1	-1 0 1
+-----+-----+	+-----+-----+
0 0 0	-2 0 2
+-----+-----+	+-----+-----+
1 2 1	-1 0 1
+-----+-----+	+-----+-----+
(Prewitt + 2x middle)	(Prewitt + 2x middle)

■ **MEMORY TRICK: "Sobel = Prewitt + 2x weighting in middle."** The 2x weighting on the central row/column gives slight smoothing -> reduces noise sensitivity. Order: Roberts < Prewitt < Sobel (size, robustness, popularity).

■ **MEMORY TRICK: "Gx detects HORIZONTAL edges. Gy detects VERTICAL edges."** Sounds backwards but: Gx = change ACROSS x = fires when intensity changes vertically (top vs bottom) = horizontal edges. Gy = vice versa.

Diagonal Edge Masks

Prewitt diagonals:

+45	-45
+-----+-----+	+-----+-----+
-1 -1 0	0 1 1
+-----+-----+	+-----+-----+
-1 0 1	-1 0 1
+-----+-----+	+-----+-----+
0 1 1	-1 -1 0
+-----+-----+	+-----+-----+
Zeros along /	Zeros along \

Sobel diagonals:

+45	-45
+-----+-----+	+-----+-----+
-2 -1 0	0 1 2
+-----+-----+	+-----+-----+

-1	0	1
0	1	2
-2	-1	0

■ **MEMORY TRICK:** "The ZEROS sit ALONG the edge direction you are detecting." +45 mask -> zeros form / line. -45 mask -> zeros form \ line. Negatives one side, positives the other.

■ NUMERICAL: Sobel - Worked Example

Q: Apply Sobel at center pixel:

10	10	10
10	20	50
10	50	80

Step 1: Compute Gx

$$G_x = (-1)(10) + (-2)(10) + (-1)(10) + (0)(10) + (0)(20) + (0)(50) + (1)(10) + (2)(50) + (1)(80)$$

$$G_x = -10 - 20 - 10 + 0 + 0 + 0 + 10 + 100 + 80$$

$$G_x = 150$$

Step 2: Compute Gy

$$G_y = (-1)(10) + (0)(10) + (1)(10) + (-2)(10) + (0)(20) + (2)(50) + (-1)(10) + (0)(50) + (1)(80)$$

$$G_y = -10 + 0 + 10 - 20 + 0 + 100 - 10 + 0 + 80$$

$$G_y = 150$$

Step 3: Magnitude & Direction

$$|\text{grad } f| = |G_x| + |G_y| = 150 + 150 = 300$$

$$\alpha = \tan^{-1}(G_y/G_x) = \tan^{-1}(1) = 45 \text{ deg}$$

Result: Strong edge of magnitude 300 at 45 degrees.

■ NUMERICAL: Practice (already solved)

Q: Apply Sobel/Prewitt to:

5	5	5
10	10	10
90	90	90

Sobel: $G_x = 340, G_y = 0, |\text{grad}| = 340, \alpha = 0 \text{ deg}$
Prewitt: $G_x = 255, G_y = 0, |\text{grad}| = 255, \alpha = 0 \text{ deg}$

Both: STRONG HORIZONTAL EDGE.

Why $G_y = 0$? Each row is uniform - no left-to-right change.

6. Laplacian & Laplacian of Gaussian (LoG)

The Laplacian (2nd derivative)

$$\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$$

Two practical Laplacian masks:

4-neighbor Laplacian

```
+---+---+---+
| 0 | -1 | 0 |
+---+---+---+
| -1 | 4 | -1 |
+---+---+---+
| 0 | -1 | 0 |
+---+---+---+
```

8-neighbor Laplacian

```
+---+---+---+
| -1 | -1 | -1 |
+---+---+---+
| -1 | 8 | -1 |  <- same as point detection!
+---+---+---+
| -1 | -1 | -1 |
+---+---+---+
```

■ **MEMORY TRICK:** "Laplacian = sum of weights ZERO. Center positive (4 or 8). Neighbors -1."

4-Laplacian uses cross neighbors. 8-Laplacian uses ALL 8 neighbors.

■■ **WARNING:** Why we DO NOT use Laplacian directly:

1. Very sensitive to noise (2nd derivative amplifies tiny variations)
2. Produces double edges (positive then negative response)
3. No directional info

Solution: Smooth FIRST with Gaussian, THEN apply Laplacian = LoG

Laplacian of Gaussian (LoG / Mexican Hat)

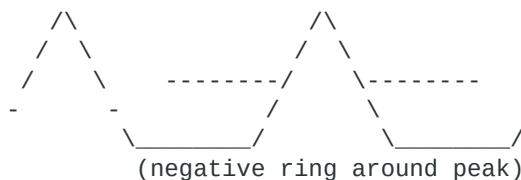
Gaussian: $h(r) = -e^{-(r^2/2\sigma^2)}$, where $r^2 = x^2 + y^2$

LoG: $\nabla^2 h(r) = -[(r^2 - \sigma^2) / \sigma^4] \cdot e^{-(r^2/2\sigma^2)}$

3D shape ("Mexican Hat" / sombrero):

3D shape:

Cross-section:



Sample 5x5 LoG mask:

```
+---+---+---+---+---+
| 0 | 0 | -1 | 0 | 0 |
+---+---+---+---+---+
| 0 | -1 | -2 | -1 | 0 |
+---+---+---+---+---+
| -1 | -2 | 16 | -2 | -1 |
+---+---+---+---+---+
| 0 | -1 | -2 | -1 | 0 |
+---+---+---+---+---+
| 0 | 0 | -1 | 0 | 0 |
+---+---+---+---+---+
```

Sum of weights = 0

■ **MEMORY TRICK:** "Gaussian smooths, Laplacian sharpens. Together: noise-resistant edge detection via zero crossings." Big σ = blurrier (ignores noise but misses fine detail). Small σ = sharper (catches detail but more noise).

■ **TIP: Derivative Theorem of Convolution:** $\partial/\partial x(h*f) = (\partial h/\partial x)*f$. This is why LoG is a single mask - we differentiate the kernel ONCE, then convolve. Saves operations.

7. Edge Linking - Local Processing

After applying Sobel/Laplacian, you get scattered edge pixels (broken, noisy). Edge linking connects them into meaningful boundaries.

Two Properties for Linking

$$|\nabla f(x,y) - \nabla f(x_{\blacksquare},y_{\blacksquare})| \leq E \text{ (similar magnitude)}$$

$$|\alpha(x,y) - \alpha(x_{\blacksquare},y_{\blacksquare})| < A \text{ (similar direction)}$$

If **both** conditions hold for adjacent pixels → **link them** as part of the same edge.

■ **MEMORY TRICK:** "Same strength + same direction = same edge. Link them." Like jigsaw puzzle: pieces only connect if colors (magnitude) AND patterns (direction) match.
E = magnitude threshold, A = angle threshold.

■ **TIP:** Real-world use: License plate detection. Letter edges have similar magnitude + direction → linking groups them into the plate region.

8. Canny Edge Detector

Canny is the **gold standard** for edge detection. 5-step algorithm.

```
+-----+
|  THE 5 STEPS OF CANNY (memorize this order)  |
+-----+
```

- | | |
|---|------------------|
| 1. Smooth image with Gaussian | (kill noise) |
| | |
| 2. Compute gradient magnitude/direction | (find edges) |
| | |
| 3. Non-maximum suppression | (thin edges) |
| | |
| 4. Double thresholding | (strong vs weak) |
| | |
| 5. Edge tracking by hysteresis | (connect) |

■ **MEMORY TRICK:** "SGTTT" - Smooth, Gradient, Thin, Threshold, Track. Sounds like "stuttt" - say it twice and you remember.

Step Details

1. Gaussian smoothing: $G * I$ - kills noise so derivatives don't amplify it.

2. Gradient: Use Sobel. Compute magnitude $|\nabla(G*I)|$ and direction $n_{\blacksquare} = \nabla(G*I) / |\nabla(G*I)|$

3. Non-maximum suppression: For each edge pixel, check 2 neighbors along gradient direction. If pixel is NOT the local max, DELETE it. Thins thick edges to 1-pixel wide.

4. Double thresholding: Pixel > T_{high} → STRONG edge (keep). Pixel < T_{low} → discard. In between → WEAK edge (decide in step 5).

5. Hysteresis tracking: Keep weak edges ONLY IF connected to a strong edge (8-neighbor). Else discard.

■ **MEMORY TRICK:** "Canny gives THIN, CONTINUOUS, NOISE-FREE edges." Gaussian → noise robust. Non-max suppression → thin edges. Hysteresis → connected, no gaps.

Non-Maximum Suppression Visualization

```
o    o    o    o
              o  <- p (interpolated)
o    o    o    o
          ^Gradient
o    o    o  <- q (edge pixel being checked)
          o  <- r (interpolated)
o    o    o    o
```

If $q > \text{both } p \text{ and } r$ -> keep q

If $q < p$ or r -> suppress (delete) q

Edge location: $\partial^2(G \cdot I) / \partial n^2 = 0$ (zero crossing of 2nd deriv along edge normal)

9. Hough Transform (CRITICAL NUMERICAL)

The Big Idea

Given scattered edge points, are any collinear? Hough Transform answers this.

In image space (xy): Line equation $y = ax + b$. A point (x,y) lies on infinitely many lines.

Switch to parameter space (ab): Rewrite as $b = -ax + y$. Now every **POINT** in image space $\rightarrow$ **LINE** in **(a,b) space**. And every **LINE** in image space $\rightarrow$ **POINT** **(a,b)** in parameter space.

IMAGE SPACE (xy)

o (x_1, y_1)
 o (x_2, y_2)
 o (x_3, y_3)
 ^
 3 collinear pts

PARAMETER SPACE (ab)

\ /
 \ / <- intersection point
 / \ gives (a,b) of
 / \ the line through
 all 3 points!

■ **MEMORY TRICK:** "Multiple collinear points in image space $\rightarrow$ multiple lines INTERSECTING AT ONE POINT in parameter space." Find the intersection $\rightarrow$ you found the line.

■ **NUMERICAL:** Hough Transform - $y = ax + b$ form (PDF SLIDES 37-40)

Q: Points A(1,4), B(2,3), C(3,1), D(4,1), E(5,0). Find the line.

Step 1: Write $b = -ax + y$ for each point

Point	(x, y)	Equation $b = -ax + y$
A	(1, 4)	$b = -a + 4$
B	(2, 3)	$b = -2a + 3$
C	(3, 1)	$b = -3a + 1$
D	(4, 1)	$b = -4a + 1$
E	(5, 0)	$b = -5a$

Step 2: Plot each line by computing (a,b) at $a=0$ and $a=1$

Point	$a = 0 \rightarrow (a, b)$	$a = 1 \rightarrow (a, b)$
A	(0, 4)	(1, 3)
B	(0, 3)	(1, 1)
C	(0, 1)	(1, -2)
D	(0, 1)	(1, -3)
E	(0, 0)	(1, -5)

Step 3: Plot all 5 lines on (a,b) graph $\rightarrow$ find intersection.

All lines (approximately) meet at **(a, b) = (-1, 5)**.

Step 4: Plug back into $y = ax + b$

$$y = -1 \cdot x + 5 \rightarrow y = -x + 5$$

Verification:

A(1,4): $y = -1+5 = 4$ YES
 B(2,3): $y = -2+5 = 3$ YES
 C(3,1): $y = -3+5 = 2$ (close, outlier)
 D(4,1): $y = -4+5 = 1$ YES
 E(5,0): $y = -5+5 = 0$ YES

■ **MEMORY TRICK:** Procedure: "For each point: write $b = -ax + y$. Plug $a=0$ and $a=1$. Plot lines. Find intersection. Read (a,b) ."

Problem with $y = ax + b$: Vertical Lines

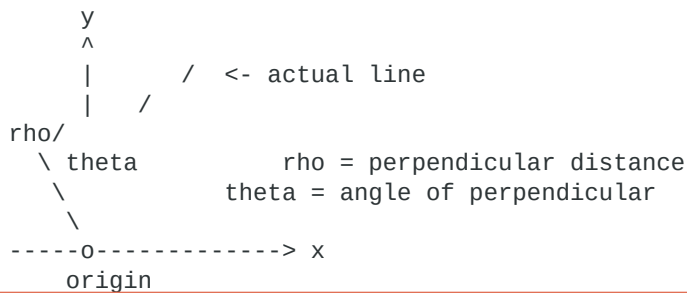
■■ **WARNING:** Vertical lines have INFINITE slope ($a \rightarrow \text{infinity}$). Computer can't store infinity. Algorithm fails for vertical lines. **Solution:** use polar form.

Solution: Polar / Normal Form

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

ρ (rho) = perpendicular distance from origin to the line.

θ (theta) = angle between this perpendicular and x-axis.



■ **MEMORY TRICK:** " ρ = how FAR. θ = which WAY. Always finite, works even for vertical lines." Each image point creates a sinusoidal curve in (ρ, θ) space. Curves intersect = line found.

■ NUMERICAL: Hough Transform - Polar form (PDF SLIDES 43-45)

Q: Points (0,5), (1,4), (3,2), (3,3), (5,0). Find the line.

Step 1: Write $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ for each

Point (x, y)	Equation
(0, 5)	$\rho = 0 \cos(\theta) + 5 \sin(\theta)$
(1, 4)	$\rho = 1 \cos(\theta) + 4 \sin(\theta)$
(3, 2)	$\rho = 3 \cos(\theta) + 2 \sin(\theta)$
(3, 3)	$\rho = 3 \cos(\theta) + 3 \sin(\theta)$
(5, 0)	$\rho = 5 \cos(\theta)$

Step 2: Vary θ from -90° to $+90^\circ$, compute ρ for each point. Produces 5 sinusoidal curves on (θ, ρ) plot.

Step 3: 4 curves intersect at approximately $(\theta = 45^\circ, \rho = 3.4)$. Point (3,3) does NOT lie on this line - it's an outlier.

Step 4: Identified line

$$x \cdot \cos(45^\circ) + y \cdot \sin(45^\circ) = 3.4$$

■ **MEMORY TRICK:** For polar form: "Vary θ , compute ρ for each point, find where sine curves intersect."

10. Thresholding

$$g(x,y) = 1 \text{ if } f(x,y) > T, \text{ else } 0$$

Types of Thresholding

Type	What	When to use
Single (Global)	One T for whole image	Histogram has 2 clear peaks
Multiple	Multiple T1, T2, ...	Histogram has 3+ peaks
Local / Adaptive	Different T per region	Uneven illumination
Optimal	T minimizes misclassification	Use probability density $p1(z)$, $p2(z)$

■ **MEMORY TRICK:** "Global = one T everywhere. Adaptive = subdivide image, T per subregion. Optimal = math-based to minimize error."

Role of Illumination

$$f(x,y) = i(x,y) \cdot r(x,y) \text{ (illumination } \times \text{ reflectance)}$$

Uneven $i(x,y)$ ruins thresholding. Adaptive thresholding (subdivision) fixes it.

Iterative Global Thresholding Algorithm

1. Pick initial T (e.g., mean intensity)
2. Segment into $G1 (>T)$ and $G2 (<=T)$
3. Compute means $\mu1$ and $\mu2$
4. New $T = (\mu1 + \mu2) / 2$
5. Repeat until T stops changing

Optimal Thresholding

Treats pixel intensities as probability density functions $p1(z)$ and $p2(z)$ (object and background). Goal: minimize probability of misclassification.

Two error types:

- Mislabeling object pixel as background
- Mislabeling background pixel as object

Boundary Characteristics & Multi-Variable

Boundary characteristics: Use gradient + Laplacian to threshold ONLY edge pixels - cleaner segmentation.

Multi-variable (Color): Threshold on R, G, B separately or jointly for color images.

11. Region-Based Segmentation - Basics

Five Conditions for Valid Segmentation

Condition	Meaning
(a) Union of $R_i = R$	Regions COVER the whole image

(b) R_i is connected	Each region is ONE PIECE (not scattered)
(c) $R_i \cap R_j = \text{empty}$	Regions DO NOT OVERLAP
(d) $P(R_i) = \text{TRUE}$	Each region is UNIFORM
(e) $P(R_i \cup R_j) = \text{FALSE}$	Adjacent regions are DIFFERENT

■ **MEMORY TRICK: "CCNUD"** - Cover, Connect, No-overlap, Uniform-each, Different-adjacent.

12. Region Growing (CRITICAL NUMERICAL)

Algorithm

1. Pick seed pixel(s)
2. Add neighbors that satisfy: $|\text{neighbor} - \text{seed}| \leq T$
3. Continue until no more pixels can be added

Connectivity: 4-neighbor (up/down/left/right) usually
or 8-neighbor (also diagonals)

■ **MEMORY TRICK: "Start with seed, grow like a plant - eat similar neighbors."** Like spreading paint from a starting drop. Paint spreads only to similar pixels.

■■ **WARNING: Important rule:** Always compare against the ORIGINAL SEED value, not the current pixel. $|\text{pixel} - \text{seed}| \leq T$. The seed value is FIXED throughout the process.

■ **TIP: Exam shortcut:**

1. Compute valid range = $[\text{seed} - T, \text{seed} + T]$
2. Highlight ALL pixels in this range
3. Keep only those CONNECTED (4-neighbor chain) to the seed
4. Done!

■ **NUMERICAL: Region Growing - Worked Example (PDF SLIDES 62-63)**

Q: Apply region growing on this 7x7 image. Seed = 60 (center), Threshold $T = 10$.

```

10 10 10 10 10 10 10
10 10 10 69 70 10 10
59 10 60 64 59 56 60
10 59 10 [60] 70 10 62    <- seed at center
10 60 59 65 67 10 65
10 10 10 10 10 10 10
10 10 10 10 10 10 10

```

Step 1: Compute valid range = $[60 - 10, 60 + 10] = [50, 70]$

Step 2: Mark all pixels in $[50, 70]$:

```

. . . . .
. . . 69 70 . .
59 . 60 64 59 56 60
. 59 . 60 70 . 62
. 60 59 65 67 . 65
. . . . .
. . . . .

```

Step 3: All marked pixels happen to be 4-connected to the seed → all included.

Final region: 19 pixels in $[50, 70]$ forming the connected segmented object. All 10s are excluded (background).

■ NUMERICAL: Practice (already solved with screenshot)

Q: 3x3 image, seed = 4, T = 2:

```
7 6 3
1 [4] 2    <- seed at center
7 5 6
```

Range: $[4-2, 4+2] = [2, 6]$

Pixels NOT in range: 7 (top-left), 7 (bottom-left), 1 (left)
All others (6, 3, 4, 2, 5, 6) are in range AND connected.

Final region (6 pixels):

```
. 6 3
. 4 2
. 5 6
```

13. Region Splitting (Quadtree) - CRITICAL NUMERICAL

Algorithm

1. Start with whole image as one region
2. If region NOT uniform (e.g., $\max - \min > T$):
-> SPLIT into 4 quadrants
3. Recurse on each quadrant
4. Stop when all regions satisfy uniformity

■ **MEMORY TRICK:** "If not uniform, chop into 4. Repeat until peaceful." Quadtree: a tree where each node has exactly 4 children.

■ NUMERICAL: Region Splitting - Worked Example (PDF SLIDE 68)

Q: 8x8 image. Threshold: $\max - \min \leq 3$ means uniform.

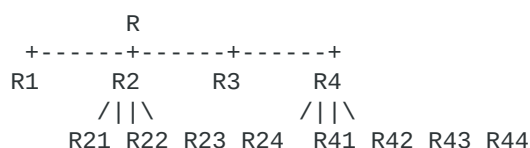
```
5 6 6 6 7 7 6 6
6 7 6 7 5 5 4 7
6 6 4 4 3 2 5 6
5 4 5 4 2 3 4 6
0 3 2 3 3 2 4 7
0 0 0 0 2 2 5 6
1 1 0 1 0 3 4 4
1 0 1 0 2 3 5 4
```

Step 1: Whole image: $\max=7$, $\min=0 \rightarrow \text{diff}=7 > 3 \rightarrow$ **SPLIT** into 4 quadrants R1, R2, R3, R4

Step 2: Check each quadrant:

```
R1 (top-left 4x4):    max=7, min=4 -> diff=3 <= 3  UNIFORM, no split
R2 (top-right 4x4):   max=7, min=2 -> diff=5 > 3    SPLIT into R21, R22, R23, R24
R3 (bottom-left 4x4): max=3, min=0 -> diff=3 <= 3  UNIFORM, no split
R4 (bottom-right 4x4): max=7, min=2 -> diff=5 > 3    SPLIT into R41, R42, R43, R44
```

Step 3: Quadtree:



14. Region Merging & Split-Merge Combined

Region Merging Algorithm

1. Start with EVERY PIXEL as its own region
2. Check adjacent regions:
If combined region satisfies uniformity -> MERGE
3. Stop when no more merging possible

■ **MEMORY TRICK:** "Splitting starts BIG and breaks down. Merging starts SMALL and combines up."

Split + Merge Combined (Smart Approach)

```
START with whole image
|
SPLIT non-uniform regions into 4 (quadtree)
|
When all regions uniform -> STOP splitting
|
MERGE adjacent regions if combined region still uniform
|
Stop when no merges possible
```

■ **TIP: Why combine both?** Pure splitting can over-segment (creates too many small regions that should be one). Merging step fixes this by joining adjacent similar regions.

Algorithm Pseudo-code

```
START: consider entire image as one region
1. If region satisfies homogeneity criteria, leave it unmodified.
2. If not, split it into four quadrants, recursively apply 1, 2
   to each newly generated region. STOP when all regions in
   the quadtree satisfy the homogeneity criterion.
3. If any two adjacent regions  $R_i$ ,  $R_j$  can be merged into a
   homogeneous region, merge them. STOP when no merging
   is possible any more.
```

15. Chain Code

Idea: Trace the boundary, write down the direction of each step.

Two Direction Systems

4-DIRECTIONAL

```
  1
  ^
2 <-+--> 0
  v
  3
```

8-DIRECTIONAL

```
      2
    3 ^ 1
4 <- + -> 0
    5 v 7
      6
```

■ **MEMORY TRICK:** "4-dir: 0=right, 1=up, 2=left, 3=down (counterclockwise from right)." 8-dir: same but with diagonals between (0-7).

How to Use

1. Pick starting point on boundary
2. Move along boundary, write direction code at each step
3. Sequence of codes = chain code

■ **NUMERICAL: Chain Code Example**

Q: Boundary going right, right, down, down, left, left, up, up (a square):

4-dir code: 0 0 3 3 2 2 1 1

16. First Difference

■■ **WARNING: Problem:** Chain code changes if you start at a different point or rotate the object. NOT invariant.

Solution: First Difference - count direction changes (counterclockwise) between adjacent codes.

Formula

First Diff = (next_code - current_code) mod 4 (for 4-dir)

First Diff = (next_code - current_code) mod 8 (for 8-dir)

■ **MEMORY TRICK:** "First difference is ROTATION INVARIANT." Rotate the object → chain code changes, but first difference stays the same.

■ **NUMERICAL: First Difference - Worked Example (PDF SLIDE 76)**

Q: Chain code: 1 0 1 0 3 3 2 2

Step: Compute pairwise (treating as circular):

```
1->0: (0-1) mod 4 = -1 mod 4 = 3
0->1: (1-0) mod 4 = 1
1->0: 3
0->3: (3-0) mod 4 = 3
3->3: 0
3->2: (2-3) mod 4 = -1 mod 4 = 3
2->2: 0
2->1: (1-2) mod 4 = -1 mod 4 = 3 (wrap-around: last -> first)
```

First Difference = 3 1 3 3 0 3 0 3

■ **TIP:** Always treat as **CIRCULAR** - the last code wraps back to compare with the first.

17. Shape Number

Definition: The first difference of **smallest magnitude** (treating as circular and rotating to find smallest).

Order n = number of digits in the sequence.

■ **MEMORY TRICK:** "Chain code → First difference → Smallest rotation = Shape Number." Order = how many digits.

■ **NUMERICAL:** Shape Number Example (PDF SLIDE 78)

Square example:

```
Chain code:      0 3 2 1
First difference: 3 3 3 3
Shape number:    3 3 3 3    (already smallest)
Order = 4
```

Larger example: First diff = 3 1 3 3 0 3 0 3

Try all rotations, pick smallest:

```
Rotation 0: 31330303
Rotation 1: 13303033
Rotation 2: 33030331
Rotation 3: 30303313
Rotation 4: 03033133  <- SMALLEST
Rotation 5: 30331330
Rotation 6: 03313303
Rotation 7: 33133030
```

```
Shape number = 03033133
Order = 8
```

18. Signature

Idea: A 1D plot of distance from centroid vs angle.

Circle: flat horizontal line at radius A
(distance constant from center)

Square: 4 peaks (corners are far from center)
and 4 dips (edge midpoints are close)

■ **MEMORY TRICK:** "Roll out the boundary into a 1D plot. Shape becomes a wave." No numerical typically. Just understand the concept.

19. Image Moments

General Formula

$$M_{ij} = \sum_x \sum_y x^i \cdot y^j \cdot I(x, y)$$

For binary images, $I(x,y) = 0$ or 1 .

Three Important Moments

M_{00} (Area):

$$M_{00} = \sum \sum I(x, y) = \text{count of 1s (for binary)}$$

M_{10} and M_{01} (used for centroid):

$$M_{10} = \sum \sum x \cdot I(x, y), M_{01} = \sum \sum y \cdot I(x, y)$$

Centroid:

$$x_{\text{bar}} = M_{10} / M_{00}, y_{\text{bar}} = M_{01} / M_{00}$$

■ MEMORY TRICK: " M_{00} = Area (count pixels). M_{10}/M_{00} = x-centroid. M_{01}/M_{00} = y-centroid."

■ NUMERICAL: Image Moments - Worked Example (PDF SLIDE 84)

Q: 4x4 binary image:

	col1	col2	col3	col4
row1:	0	0	0	0
row2:	1	1	1	1
row3:	1	1	1	1
row4:	0	0	0	0

M_{00} = count of 1s = 8 (area)

M_{10} = sum of (row * value) for all 1s:

Row 2 has four 1s -> 2+2+2+2 = 8
 Row 3 has four 1s -> 3+3+3+3 = 12
 $M_{10} = 8 + 12 = 20$

M_{01} = sum of (col * value) for all 1s:

Col 1: rows 2,3 -> 1+1 = 2
 Col 2: rows 2,3 -> 2+2 = 4
 Col 3: rows 2,3 -> 3+3 = 6
 Col 4: rows 2,3 -> 4+4 = 8
 $M_{01} = 2+4+6+8 = 20$

Centroid:

$x_{\text{bar}} = 20 / 8 = 2.5$
 $y_{\text{bar}} = 20 / 8 = 2.5$

Centroid = (2.5, 2.5)
 (Makes sense - center of the white block)

Central Moments (translation invariant)

$$\mu_{pq} = \sum \sum (x - x_{\text{bar}})^p (y - y_{\text{bar}})^q I(x, y)$$

Useful identities:

- $\mu_{00} = M_{00}$ (same as regular)
- $\mu_{10} = 0$
- $\mu_{01} = 0$

20. MASTER CHEAT SHEET

All Operators at a Glance

Operator	Order	Mask Size	Detects	Noise Sens.
Roberts	1st deriv	2x2	Diagonal edges	High
Prewitt	1st deriv	3x3	H/V edges	Medium
Sobel	1st deriv	3x3	H/V edges	Lower
Laplacian	2nd deriv	3x3	All edges	Very High
LoG (Mexican Hat)	2nd deriv	5x5+	Edges (zero cross)	Low
Canny	Multi-step	Variable	Best general	Very Low

Region Methods Comparison

Method	Direction	Start	Stop
Growing	Bottom-up	Seed pixel	No more similar neighbors
Splitting	Top-down	Whole image	All regions uniform
Merging	Bottom-up	Every pixel	No more merges possible
Split-Merge	Both	Whole image	Uniform AND no merges

Boundary Descriptors Recap

Descriptor	What it gives	Key Trick
Chain Code	Boundary as direction sequence	4 or 8 directions
First Difference	Rotation-invariant version	(next - current) mod 4 or 8
Shape Number	Smallest rotation of first diff	Order = digit count
Signature	1D distance-vs-angle plot	Visual signature of shape
Image Moments	Statistical shape descriptors	M00=area, centroid via M10/M00
Central Moments	Translation-invariant moments	$\mu_{10} = \mu_{01} = 0$

Master Mnemonics Collection

Topic	Mnemonic
Segmentation split	"Discontinuity finds borders, similarity finds interiors."
Point detection	"Center 8, neighbors -1. Quiet on flat, screams at lonely pixels."
Line detection	" +2 along the line direction, -1 elsewhere."
Edge derivatives	"1st deriv -> peak. 2nd deriv -> zero crossing."
Sobel vs Prewitt	"Sobel = Prewitt + 2x weighting in middle."

Gx vs Gy	"Gx detects horizontal edges, Gy detects vertical."
LoG	"Gaussian smooths, Laplacian sharpens. Mexican Hat = both."
Canny	"SGTTT - Smooth, Gradient, Thin, Threshold, Track."
Hough idea	"Points in image space -> lines in parameter space. Intersection = real line."
Polar Hough	"rho = how FAR. theta = which WAY. Always finite."
Region grow	"Compare against the SEED, not the current pixel. Range [seed-T, seed+T]."
Quadtree	"If not uniform, chop into 4. Repeat until peaceful."
5 conditions	"CCNUD - Cover, Connect, No-overlap, Uniform-each, Different-adjacent."
Chain code	"4-dir: 0=right, 1=up, 2=left, 3=down (counterclockwise)."
First diff	"(next - current) mod 4. Treat as CIRCULAR. Rotation invariant."
Shape number	"First difference, rotated to smallest magnitude."
Image moments	"M00=area (count). Centroid: M10/M00 and M01/M00."

21. Numerical Practice Checklist

Top-Priority Numericals (Practice these tonight!)

#	Numerical	Difficulty	Likelihood
1	Hough Transform (a, b form)	Hard	■■■■
2	Hough Transform (rho, theta form)	Hard	■■■■
3	Region Growing	Medium	■■■■
4	Region Splitting (Quadtree)	Medium	■■■■
5	Sobel/Prewitt Gradient	Medium	■■■■
6	Point Detection	Easy	■■
7	Line Detection	Easy	■■
8	Image Moments (Centroid)	Easy	■■
9	First Difference	Easy	■■
10	Shape Number	Easy	■■

Quick Numerical Procedures (Cheat Sheet)

Hough Transform ($y = ax + b$):

1. For each point (x, y) , write: $b = -ax + y$
2. Plug $a=0$ and $a=1$ into each equation, get (a, b) pairs
3. Plot lines on (a, b) plane
4. Find intersection point $\rightarrow$ that is your (a, b)
5. Final line: $y = ax + b$

Hough Transform (polar form):

1. For each point (x, y) , write: $\rho = x \cos(\theta) + y \sin(\theta)$
2. Vary θ from -90 to $+90$ degrees
3. Plot sinusoidal curves on (θ, ρ) plane
4. Find intersection $\rightarrow (\theta, \rho)$
5. Final line: $\rho = x \cos(\theta) + y \sin(\theta)$

Region Growing:

1. Compute valid range = $[\text{seed} - T, \text{seed} + T]$
2. Highlight ALL pixels in range
3. Keep only those connected (4-neighbor) to seed
4. Reject disconnected pixels

Region Splitting:

1. Compute $(\max - \min)$ of region
2. If $>$ threshold: split into 4 quadrants
3. Recurse on each quadrant
4. Stop when all regions have $(\max - \min) \leq$ threshold
5. Draw quadtree

Sobel/Prewitt Gradient:

1. Apply G_x mask $\rightarrow$ sum of weighted pixels
2. Apply G_y mask $\rightarrow$ sum of weighted pixels
3. Magnitude = $|G_x| + |G_y|$
4. Direction = $\tan^{-1}(G_y / G_x)$

Image Moments:

For binary image:

M_{00} = count of 1s (area)

M_{10} = sum of x-coordinates of all 1s

M_{01} = sum of y-coordinates of all 1s

Centroid = (M_{10}/M_{00} , M_{01}/M_{00})

22. Likely Exam Questions

Theory Questions (Short Answer)

1. Define image segmentation. What are its two approaches?
2. Explain point detection with mask and example.
3. Write the four line detection masks.
4. Differentiate ideal edge from ramp edge.
5. Compare 1st and 2nd derivative-based edge detection.
6. Compare Roberts, Prewitt, and Sobel operators.
7. What is Sobel's advantage over Prewitt?
8. Why is LoG called "Mexican Hat"? Derive its expression.
9. Explain Canny edge detector with all 5 steps.
10. Why is non-maximum suppression needed in Canny?
11. Why use polar form (ρ , θ) instead of (a , b) in Hough Transform?
12. Compare global vs adaptive thresholding.
13. Explain iterative algorithm for optimal threshold.
14. State the 5 conditions for region-based segmentation (CCNUD).
15. Compare region growing vs region splitting.
16. Explain split-merge combined approach.
17. What is chain code? Explain 4-dir and 8-dir.
18. Why is first difference rotation invariant?
19. Define shape number and its order.
20. What is signature? Sketch for circle and square.
21. Define image moments. What is the centroid in terms of moments?

Numerical Questions (Likely)

1. Apply point detection on given 3x3 with given threshold T .
2. Apply line detection masks on given neighborhood, find direction.
3. Compute Sobel/Prewitt gradient (G_x , G_y , magnitude, direction).
4. Apply Hough transform on given set of points ($y = ax+b$ form).
5. Apply Hough transform with polar form for given points.
6. Apply region growing with given seed and threshold.
7. Apply region splitting (quadtree) with given threshold.
8. Generate chain code for given boundary (4-dir or 8-dir).
9. Compute first difference and shape number from given chain code.
10. Compute area and centroid of given binary image using moments.

ALL THE BEST!

You've covered Module 4 completely.

Trust your prep. Read this once more, do PYQs, and walk in confident.

Key strategy for the exam:

1. Do the easy numericals FIRST (Sobel, Image Moments, Chain code)
2. Then the medium ones (Region growing, Region splitting)
3. Save Hough Transform for when you have a clear head
4. For theory: Always lead with the memory trick / mnemonic - it organizes your answer
5. Draw masks/diagrams - examiners love visuals